

Milestone 2: Tổng kết Toàn diện Bức tranh eShop

Đúc kết 35 Video Thực hành & Bộ phóng
Kiến trúc Doanh nghiệp

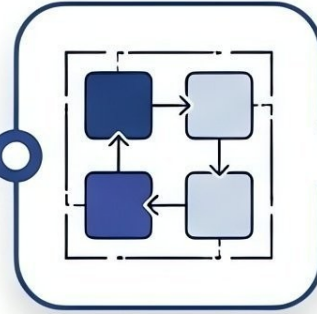
- ✓ **Yêu cầu Tiên quyết:** Đã hoàn thiện UI, Blazor In-Depth (Part 1-2) và Kiến trúc, SQL Dapper (Part 3-5).
- ✓ **Mục tiêu Trọng tâm:** Làm chủ ranh giới 4 tầng Clean Architecture.
- ✓ **Kết quả Đầu ra:** Sẵn sàng nền tảng vững chắc cho Đồ án Capstone.



Hành trình 35 Video: Từ Memory đến SQL Server



Chặng 1 (Video 01-20)
Khởi tạo UI & Component
(In-Memory).



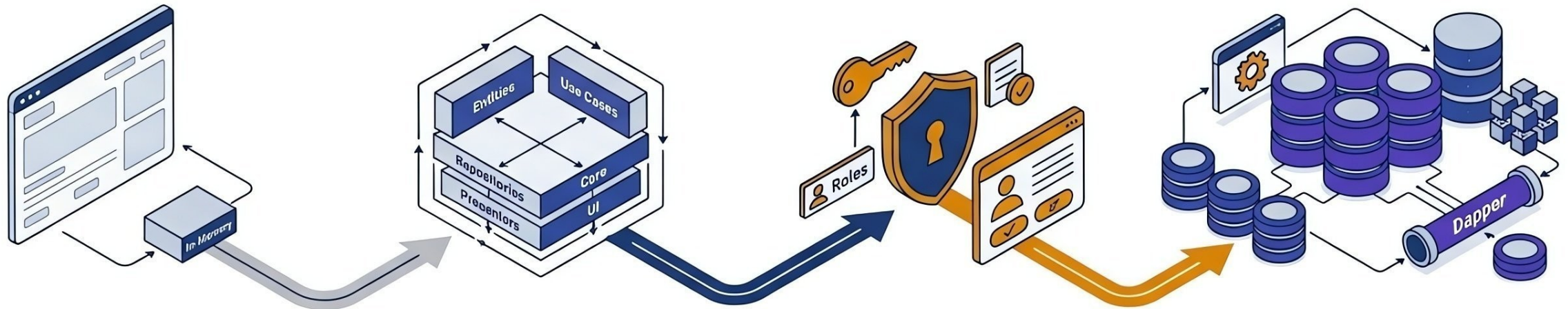
Chặng 2 (Video 21-30)
Tách 4 Project Clean
Architecture.



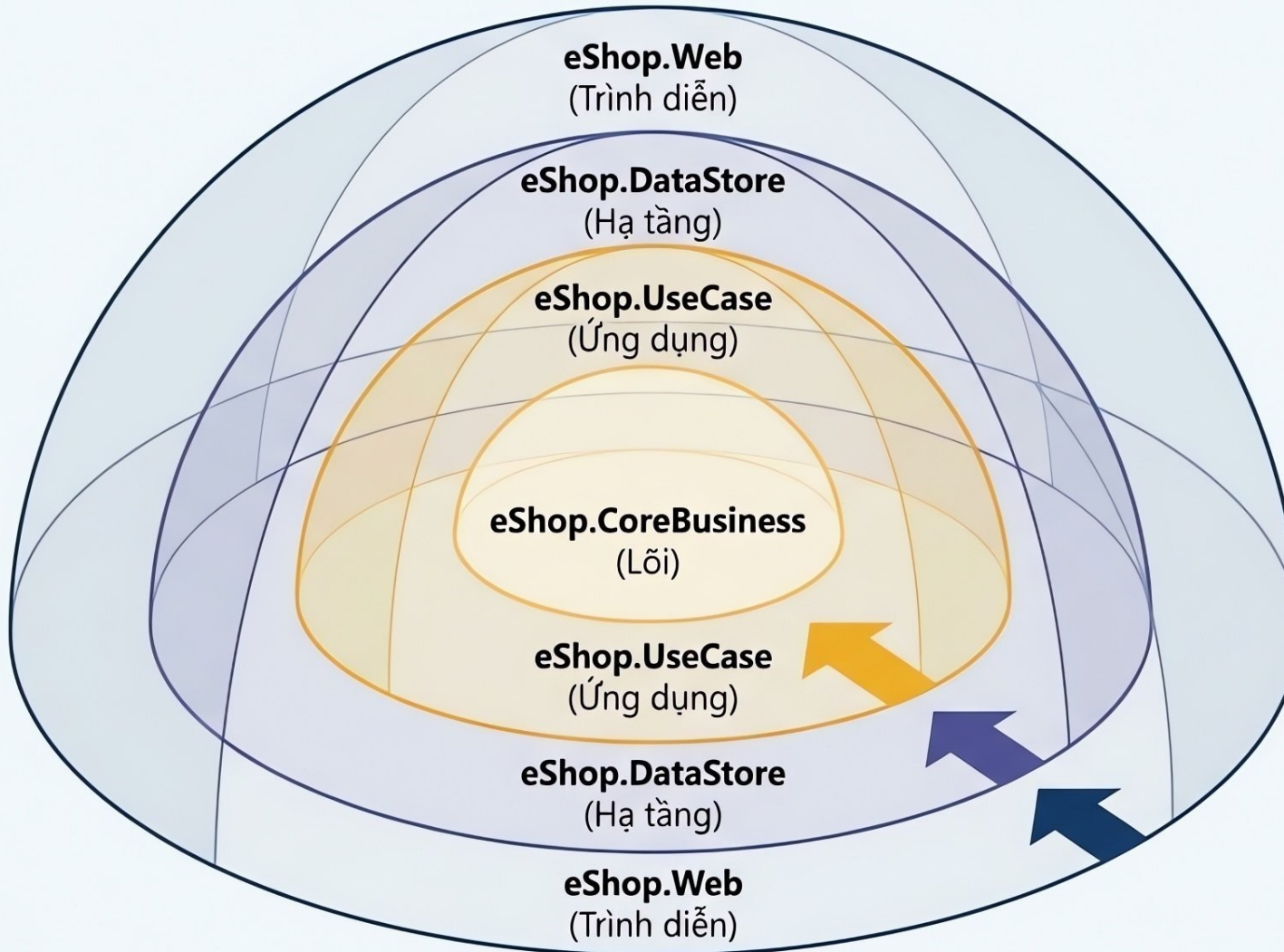
Chặng 3 (Video 31-33)
Phân quyền &
Authentication.



Chặng 4 (Video 34-35)
Kết nối SQL Server
(Dapper).



Giải phẫu Tổng thể 4 Tầng Clean Architecture



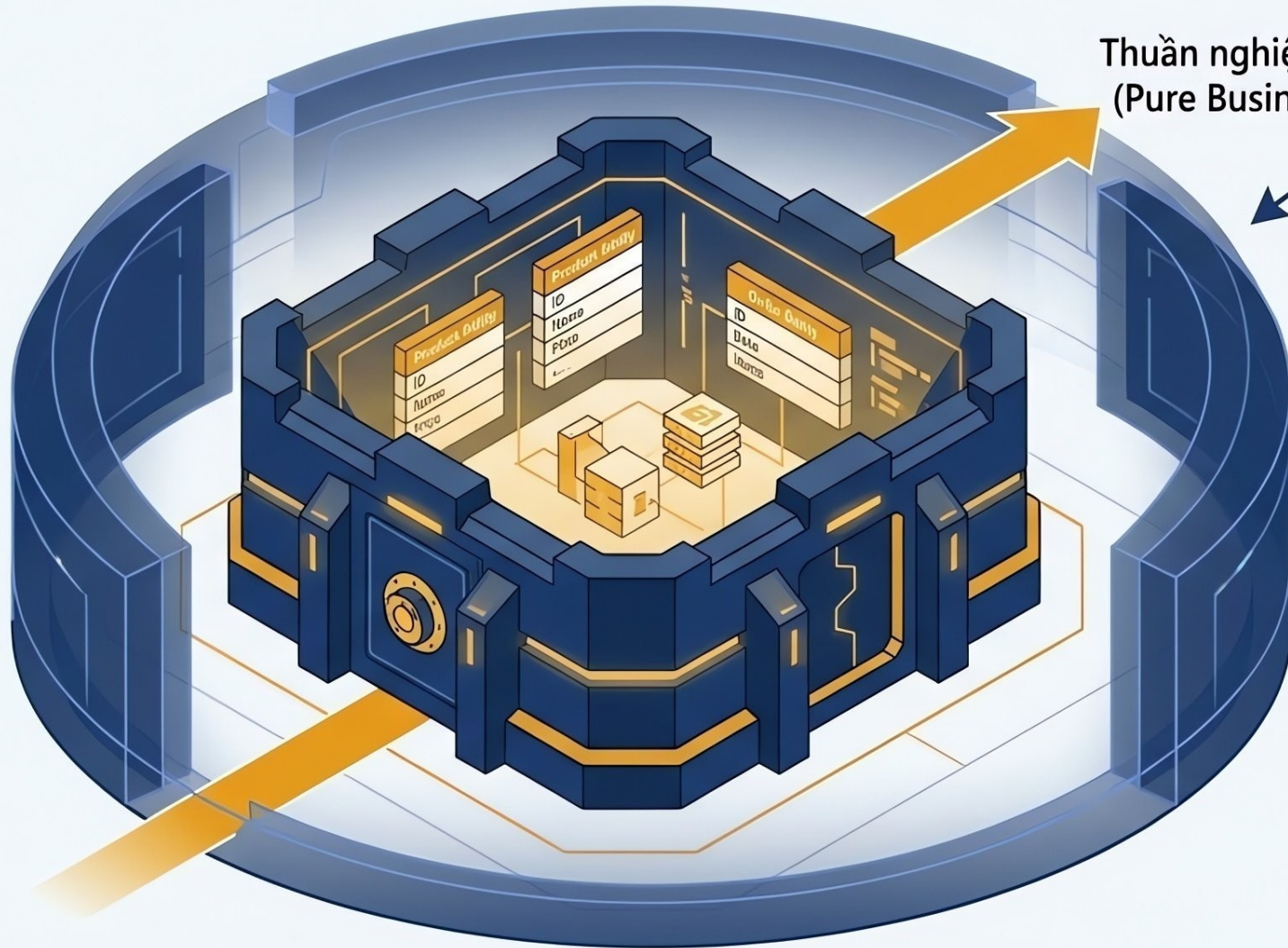
1. eShop.CoreBusiness (Lõi)
Thực thể nghiệp vụ thuần túy.

2. eShop.UseCase (Ứng dụng)
Điều phối luồng xử lý.

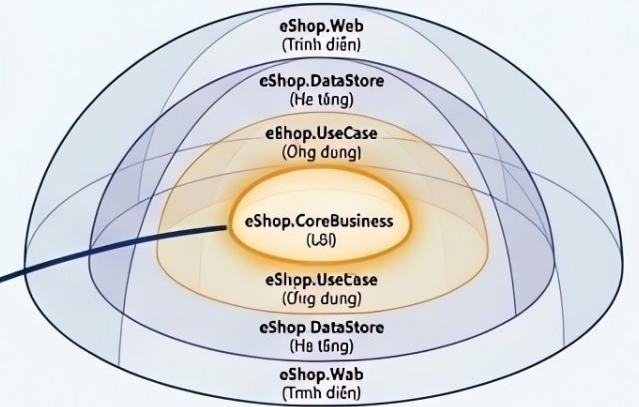
3. eShop.DataStore (Hạ tầng)
Plugin lưu trữ linh hoạt.

4. eShop.Web (Trình diễn)
Giao diện Blazor UI.

Tầng 1 — CoreBusiness: Trái tim Nghiệp vụ



Thuần nghiệp vụ
(Pure Business)



Vai trò Thiết kế

Lưu giữ quy tắc nghiệp vụ bất biến
(Product, Order).



Quy tắc Vàng (Zero Dependencies)

Tuyệt đối không tham chiếu thư viện ngoài.

Tính Độc lập

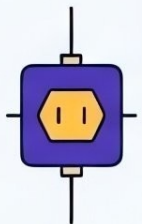
Có thể di chuyển sang Mobile/Desktop App mà không cần sửa code.

Tầng 2 — UseCase: Nhạc trưởng Điều phối



Điều phối Nghiệp vụ

Chứa các kịch bản cụ thể
(Ví dụ: ProcessOrderUseCase).

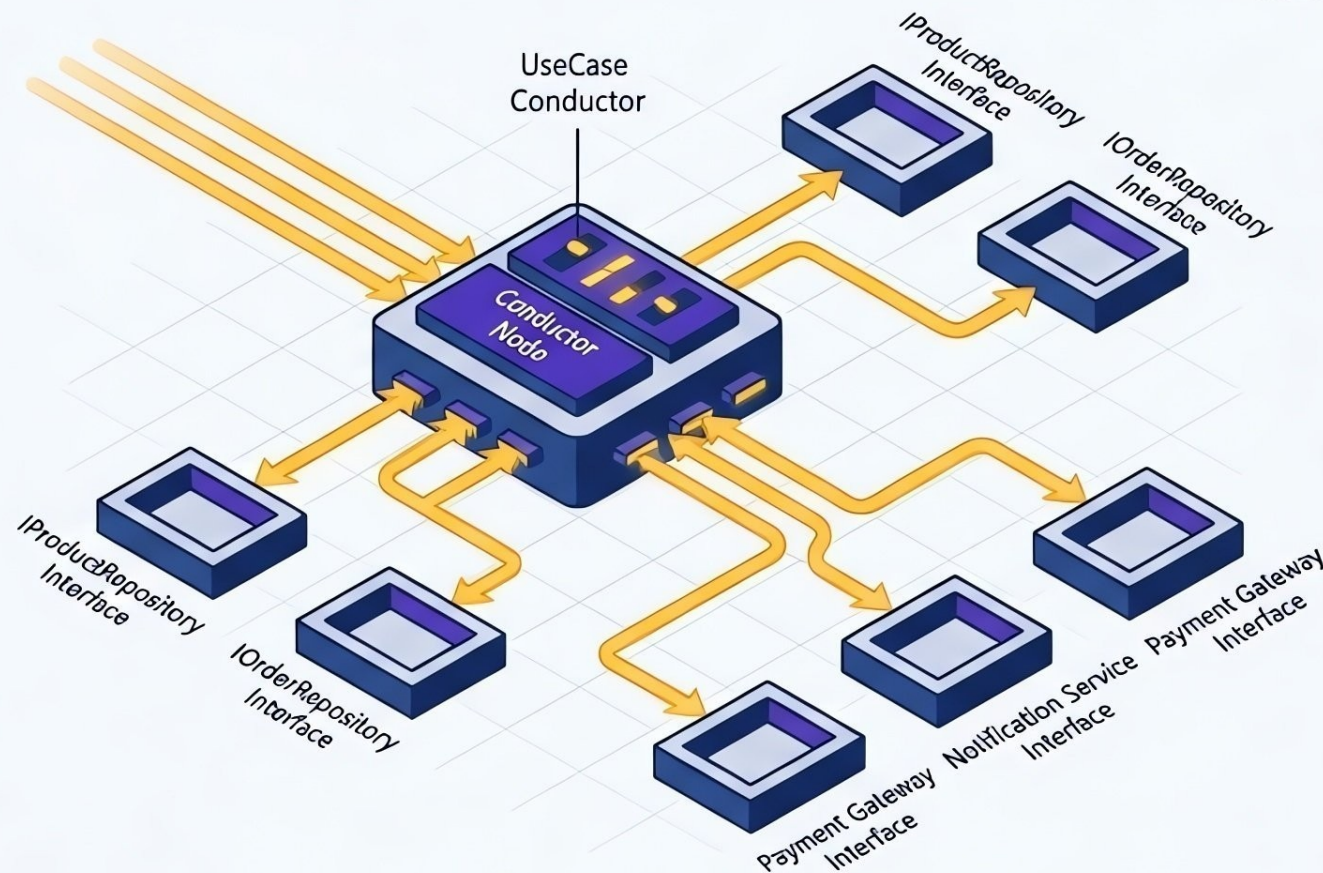


Định nghĩa Hợp đồng (PluginInterfaces)

Nơi khai báo các interfaces
(IProductRepository,
IOrderRepository).

Nguyên tắc Phụ thuộc

Chỉ tham chiếu duy nhất tới tầng
CoreBusiness.



Orchestrating flows between business rules and external contracts.

Tầng 3 — DataStore: Cơ chế Cắm rút (Pluggable)

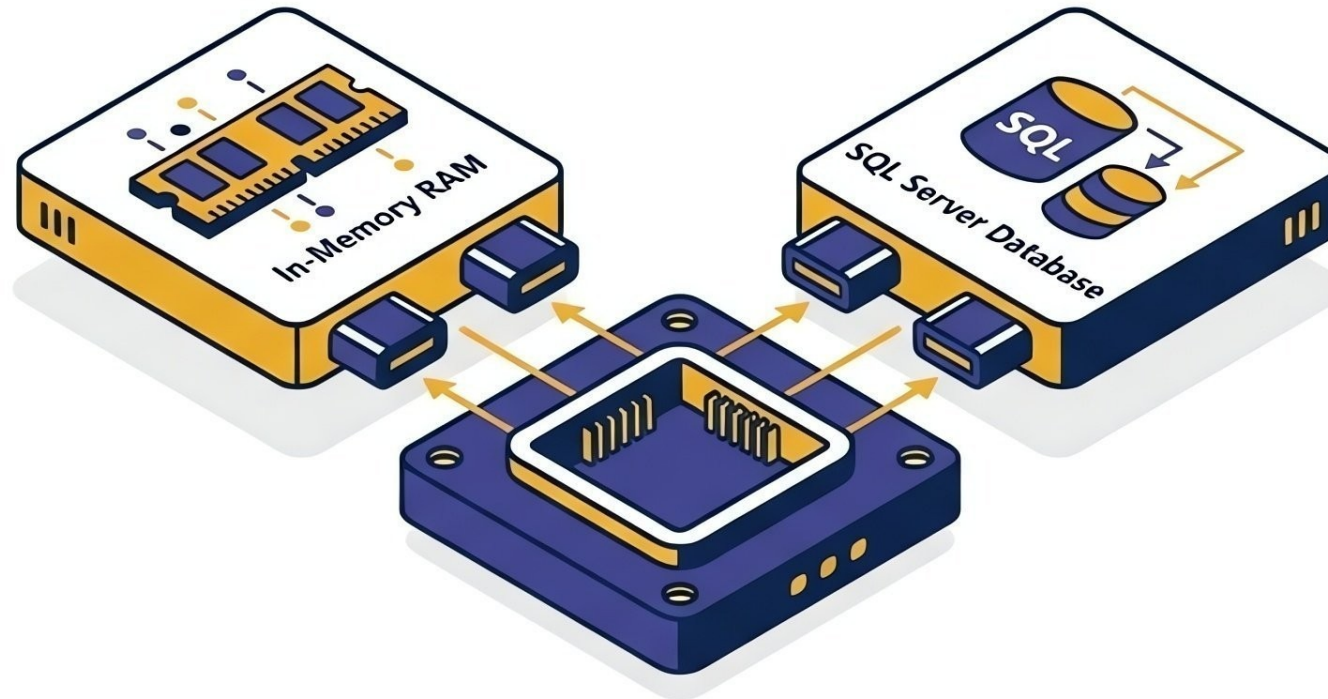


eShop.DataStore.HardCoded

Lưu trữ danh sách tạm trên bộ nhớ (Lập trình nhanh).

eShop.DataStore.SQL

Kết nối SQL Server qua thư viện Dapper (Môi trường thực tế).



Chuyển đổi giữa 2 môi trường chỉ bằng 1 dòng cấu hình Dependency Injection trong Program.cs.

Tầng 4 — **Web**: Tầng Trình diễn Blazor Server



Tổ chức UI

- Thư mục Pages/ (Màn hình) và Controls/ (Linh kiện tái sử dụng).

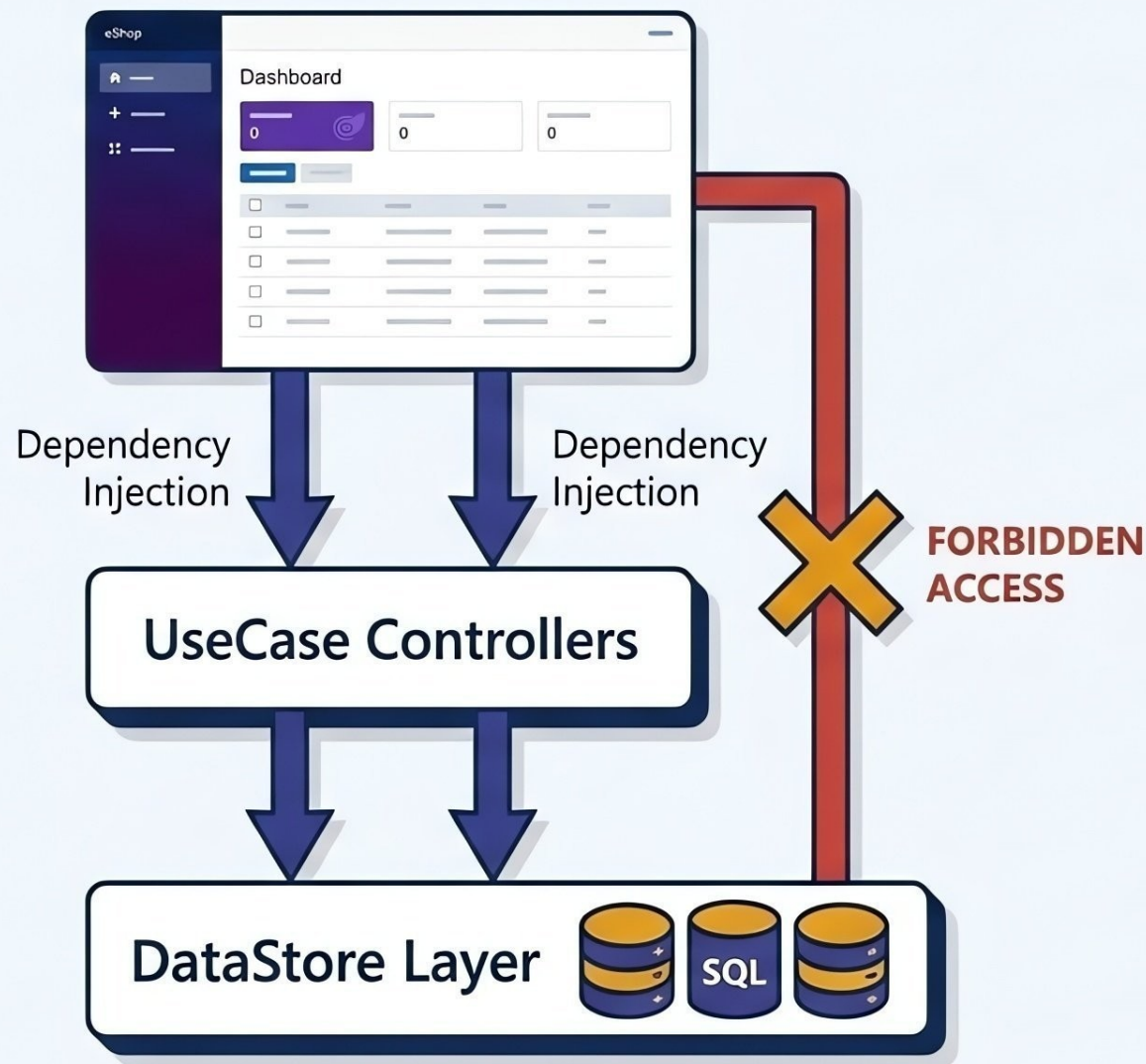
Trung tâm Kết nối

- Program.cs cấu hình toàn bộ 4 tầng.

Quy tắc Gọi lệnh Bắt buộc



Giao diện chỉ được @inject lớp UseCase, tuyệt đối không truy cập trực tiếp DataStore.



Cuộc chiến ORM: Dapper vs. EF Core

 Dapper (Micro-ORM)	Entity Framework Core (Heavy ORM)
⋮ Đặc điểm: Gọn nhẹ, ánh xạ trực tiếp SQL sang C#.	DAPPER  EF CORE  ⋮ Đặc điểm: Tự động sinh SQL, hỗ trợ Migration.
⌚ Hiệu năng: Tốc độ cực cao, tương đương ADO.NET thuần.	⌚ Hiệu năng: Tiêu tốn nhiều tài nguyên bộ nhớ hơn.
📌 Phù hợp: Kiểm soát 100% SQL, hệ thống tài chính thương mại phức tạp.	📌 Phù hợp: Phát triển nhanh các tính năng CRUD cơ bản.



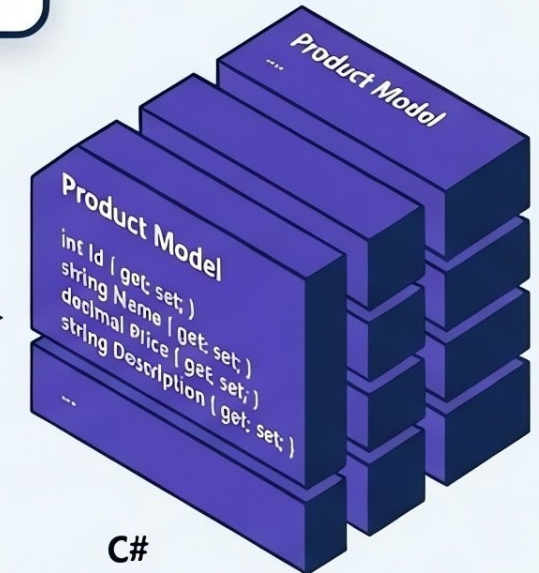
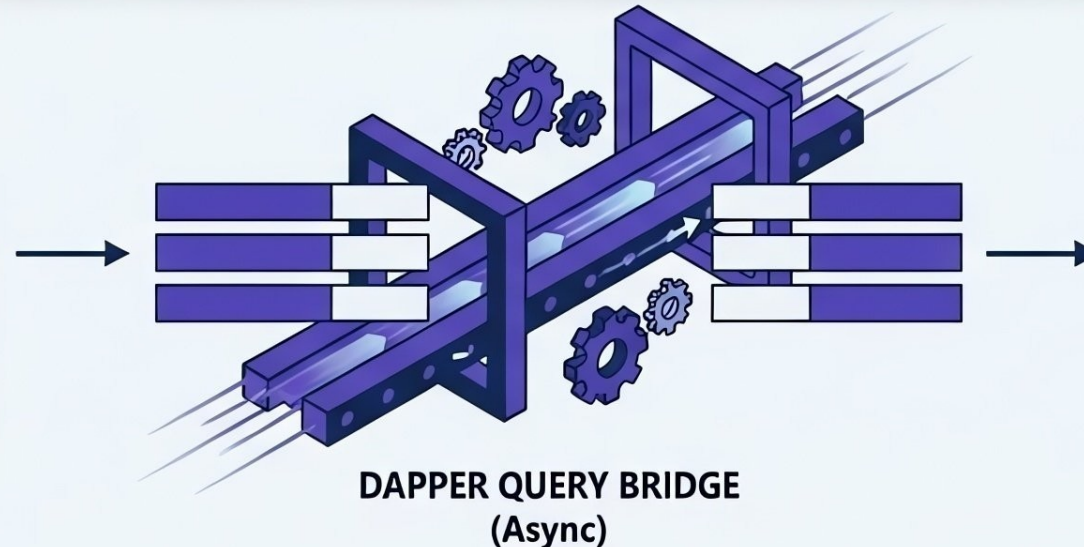
Mở xẻ Cơ chế Hoạt động của Dapper

Bộ Ba Thao tác Cốt lõi:

- **QueryAsync<T>**: Trích xuất danh sách đối tượng.
- **QueryFirstOrDefaultAsync<T>**: Lấy chi tiết 1 bản ghi duy nhất.
- **ExecuteAsync**: Thực thi thao tác thay đổi (INSERT, UPDATE, DELETE).

Id	Name	Price	Description
101	"Product A"	99.99	...
102	"Product B"	127.99	...
103	"Product C"	159.99	...
104	"Product D"	99.99	...
105	"Product E"	99.99	...

STRUCTURED SQL RELATIONAL
TABULAR DATA

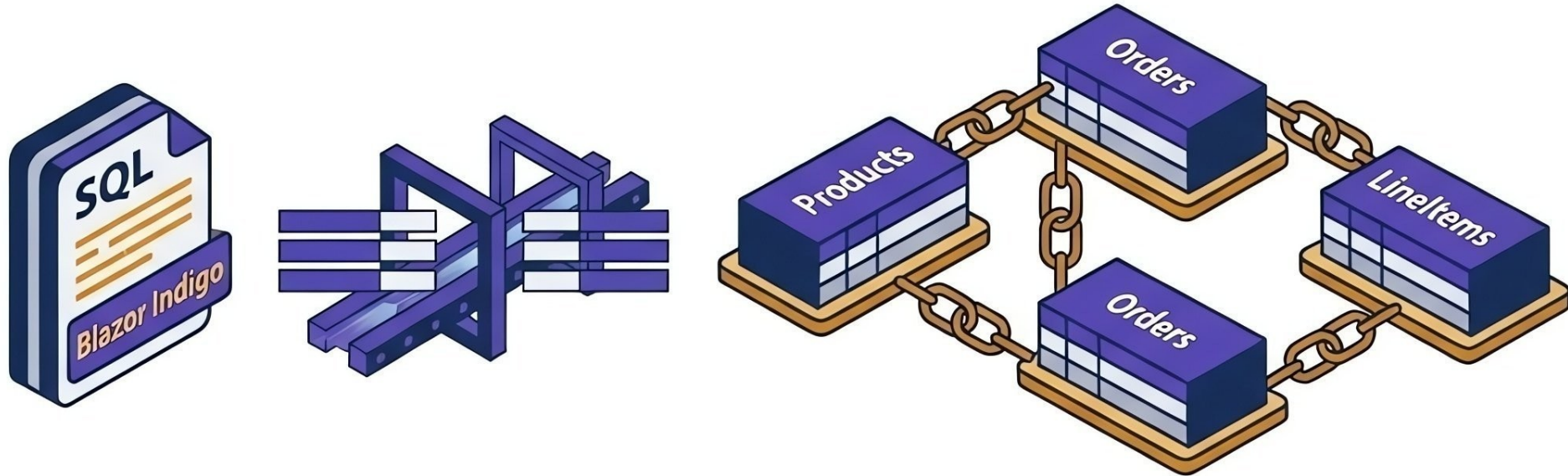


C#

Tự động Ánh xạ (Auto-Mapping)

Tên cột SQL tự động khớp chính xác với thuộc tính của Model C#, tiết kiệm 80% mã nguồn thủ công.

Khởi tạo Dữ liệu: Script eShop.SchemaAndData.sql



1. Cấu trúc Bảng



Tạo Products và Orders kèm các ràng buộc khóa ngoại (Foreign Key) chuẩn mực.

2. Thực thi (Execute)



Mở SSMS → Kết nối CSDL → F5.

3. Dữ liệu Mẫu (Seed Data)

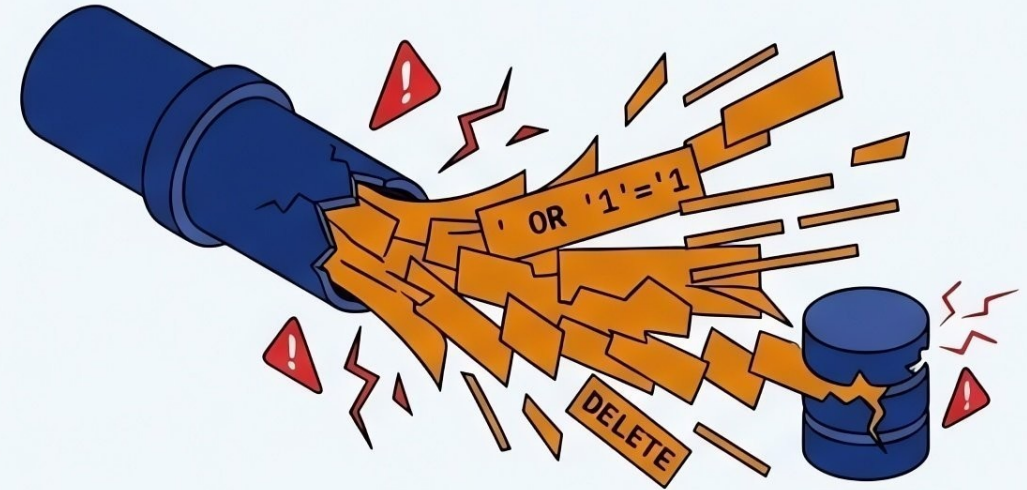


Hệ thống tự động chèn sẵn hàng loạt sản phẩm để phục vụ kiểm thử ngay lập tức.

Bảo mật Dữ liệu: Chống SQL Injection

Thảm họa Nối chuỗi

Nguy cơ: Lệnh `SELECT * FROM Products WHERE Id = " + userInput`; có thể bị hacker lợi dụng bằng cú pháp `' OR '1'='1` để đánh cắp/xóa dữ liệu.



Giải pháp Parameterized Query (Bắt buộc)

Thực hành: Dùng tham số `@Id` (VD: `WHERE Id = @Id`).

Kết quả: Dapper tự động làm sạch dữ liệu, vô hiệu hóa 100% âm mưu chèn mã độc.



Quản lý Chuỗi Kết nối (Connection String)

Lưu trữ Tiêu chuẩn

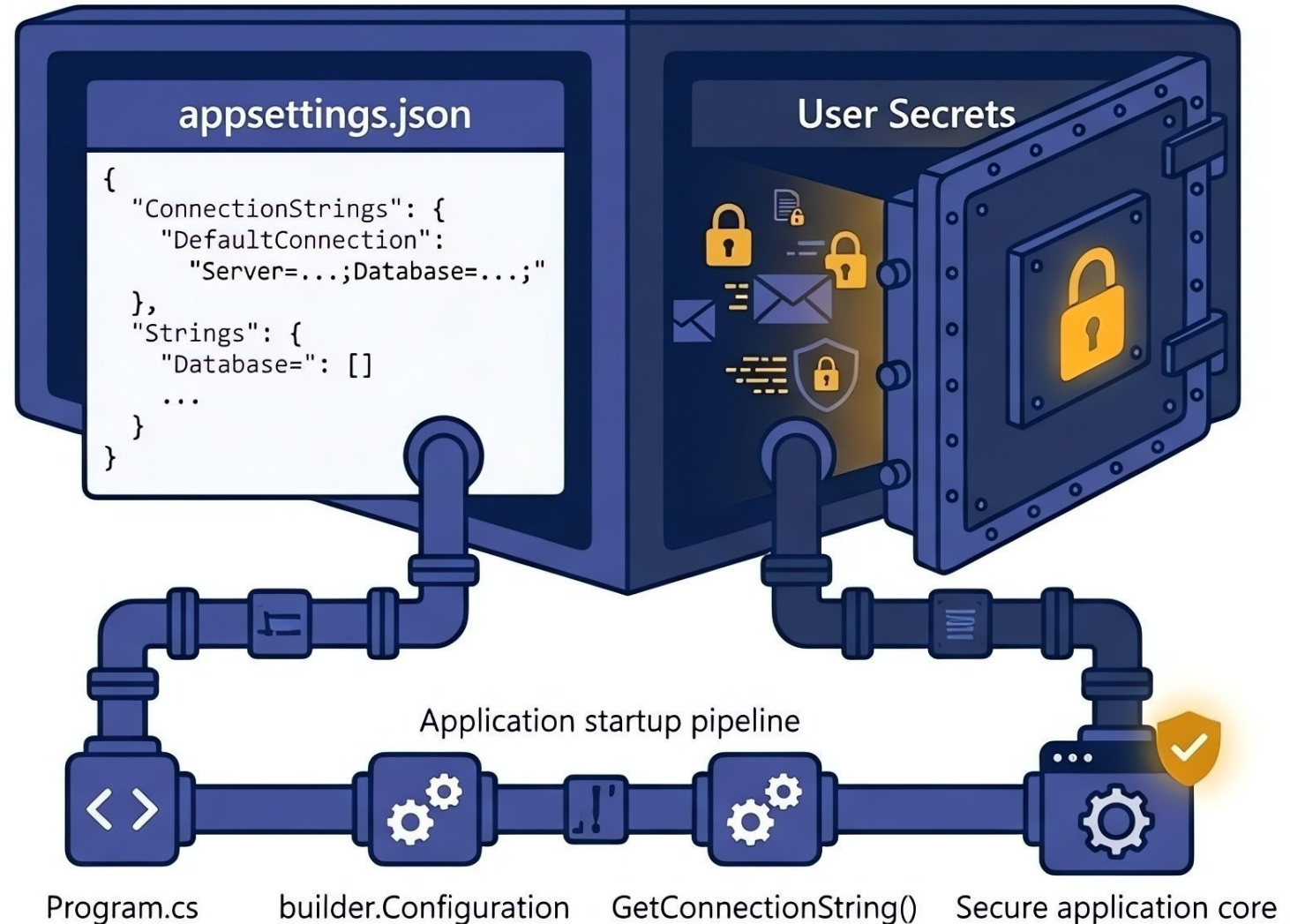
Khai báo chuỗi kết nối an toàn tại file `appsettings.json`.

Đọc Cấu hình Khởi động

Ứng dụng nạp chuỗi thông qua `builder.Configuration.GetConnectionString()` trong `Program.cs`.



Sử dụng **User Secrets** để tránh nguy cơ rò rỉ mật khẩu CSDL lên GitHub công khai.

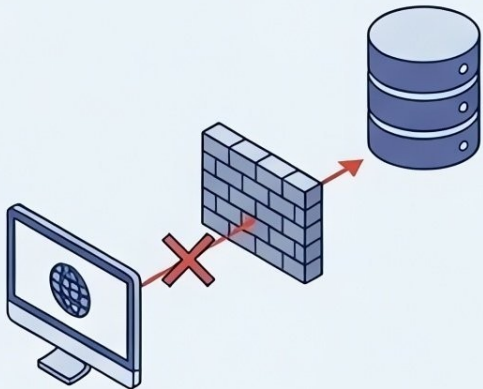


Bảng Chẩn đoán: Top 3 Lỗi Kết nối Phổ biến

Cấu hình & Mạng (Network)



red light

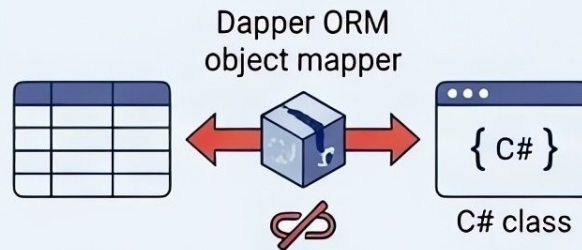


Thường do thiếu cờ
TrustServerCertificate=True, sai tên máy
chủ (.\\SQLEXPRESS), hoặc chưa mở
cổng 1433.

Ánh xạ Dữ liệu (Mapping)



red light



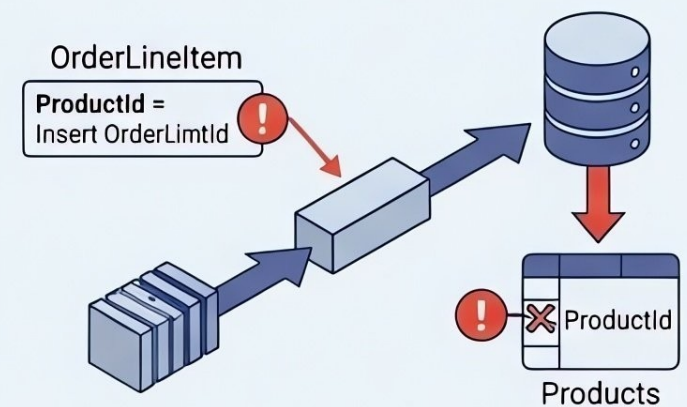
Dapper trả về rỗng do tên cột SQL không
khớp chính xác với thuộc tính C#.

SQL Column	C# Property
User_Id	UserId
!	!
MISMATCH	

Ràng buộc (Constraints)

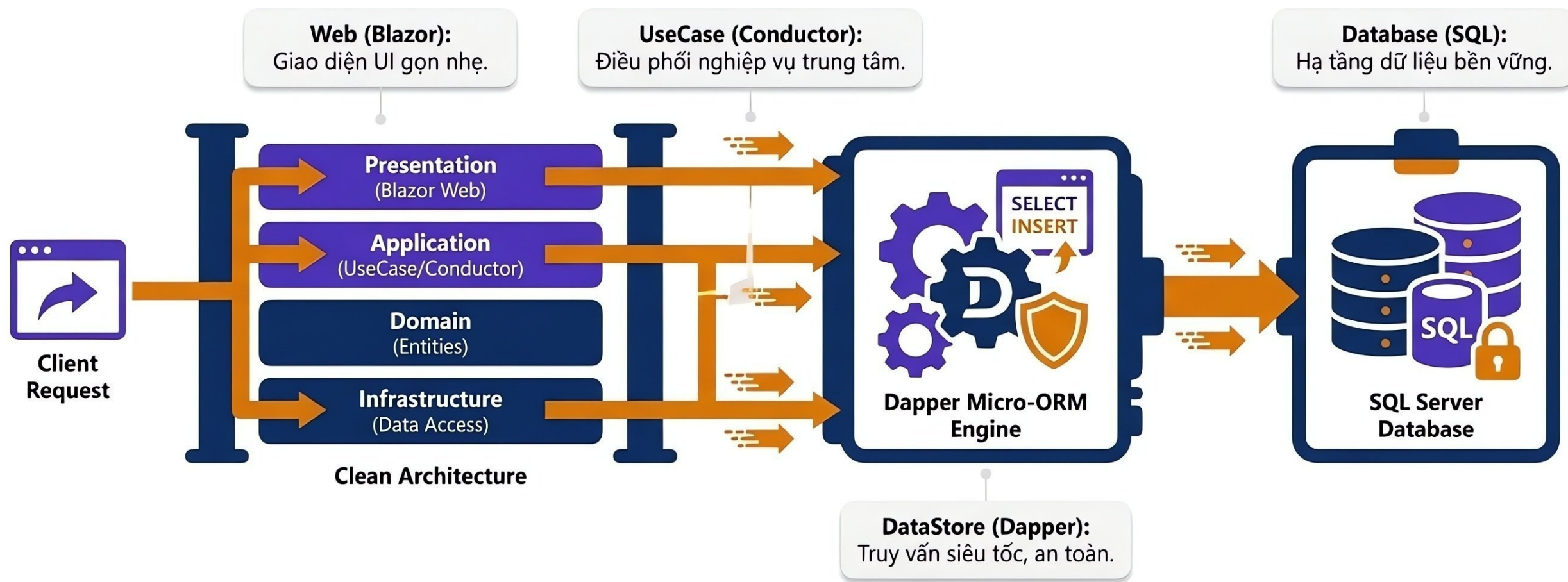


red light



Vi phạm Khóa ngoại (VD: Cố thêm
OrderItem với ProductId không tồn tại).

Tổng kết Part 1: Bức tranh Hệ thống Hoàn chỉnh



Nền tảng Backend đã sẵn sàng 100% theo tiêu chuẩn Doanh nghiệp.

Thách thức Tiếp theo: Bước đệm tới Capstone



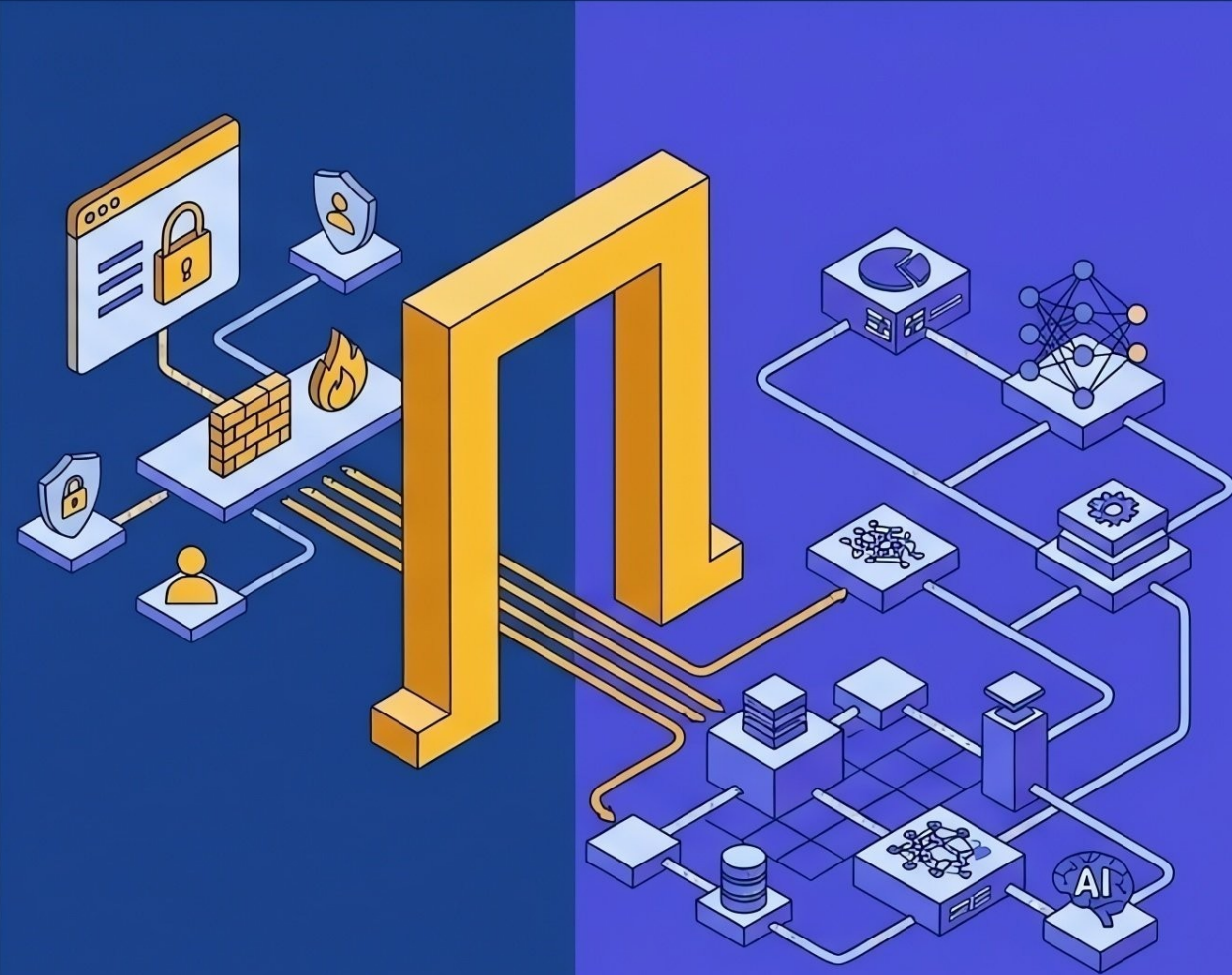
Thực trạng: Hệ thống eShop vận hành trơn tru nhưng **Admin Portal** đang bị mở toang, thiếu bảo mật.



Bài toán Đặt ra: Khóa chặt phân hệ bằng **Cookie Authentication** như thế nào?



Giai đoạn Tiếp theo (Part 2): Khám phá bảo mật phân quyền, Ma trận 10 Miền nghiệp vụ độc bản và tích hợp **Trợ lý AI** cho Đề án Capstone.



Khóa chặt bảo mật, định hướng đồ án và mở rộng công nghệ AI



Trọng tâm bảo mật

Phân quyền quản trị
Admin Portal với Cookie
Authentication.



Chiến lược đồ án

Chuyển dịch eShop sang
10 bài toán kinh doanh
độc bản.



Đón đầu tương lai

Tích hợp AI doanh
nghiệp thế hệ mới.



Phân biệt bản chất giữa Xác thực và Phân quyền trong hệ thống

Xác thực (Authentication)

Bạn là ai?

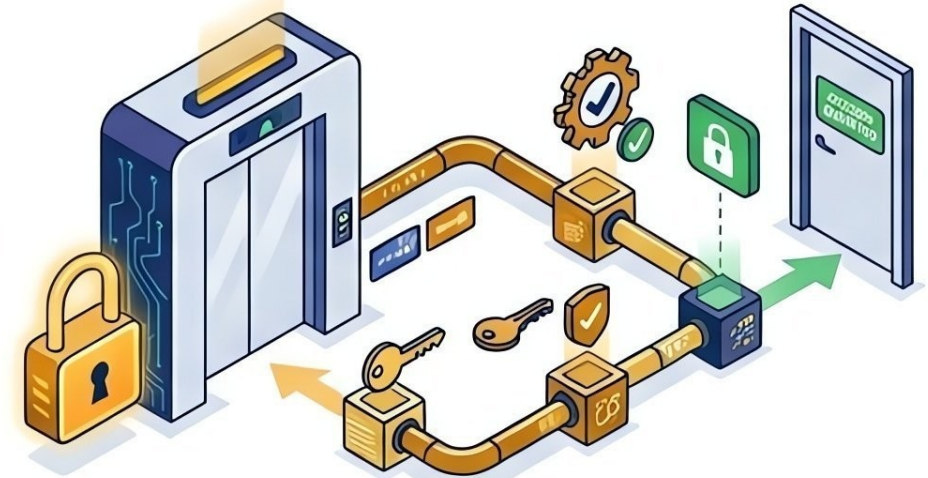


Cơ chế: Kiểm tra danh tính người dùng thông qua Tên đăng nhập và Mật khẩu (hoặc mã OTP).

Kết quả: Người dùng này có tài khoản hợp lệ trong hệ thống hay không?

Phân quyền (Authorization)

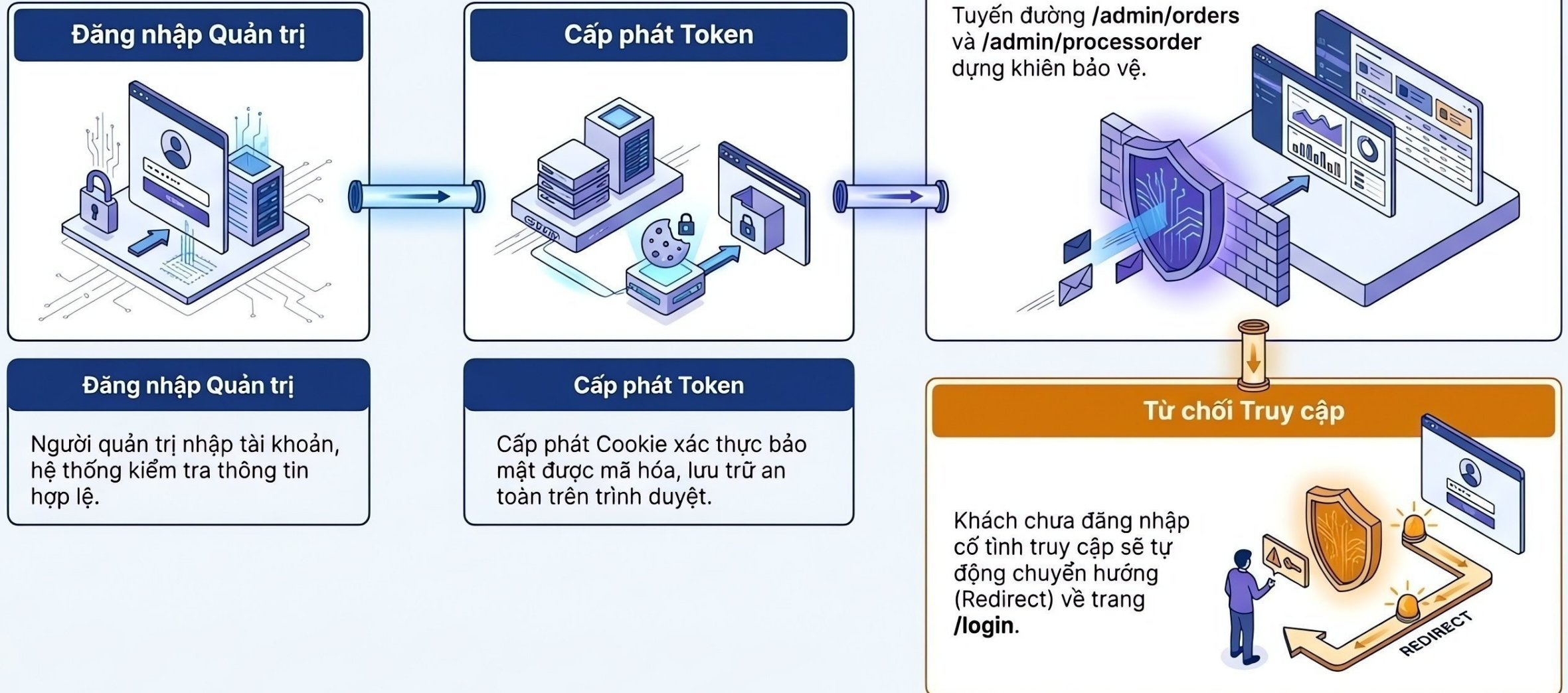
Bạn được phép làm gì?



Cơ chế: Kiểm tra vai trò (Roles) và quyền hạn (Policies) của người dùng đã xác thực.

Kết quả: Khách hàng có được phép vào trang Quản trị Đơn hàng hay không?

Cơ chế bảo vệ tuyến đường quản trị qua Cookie Authentication

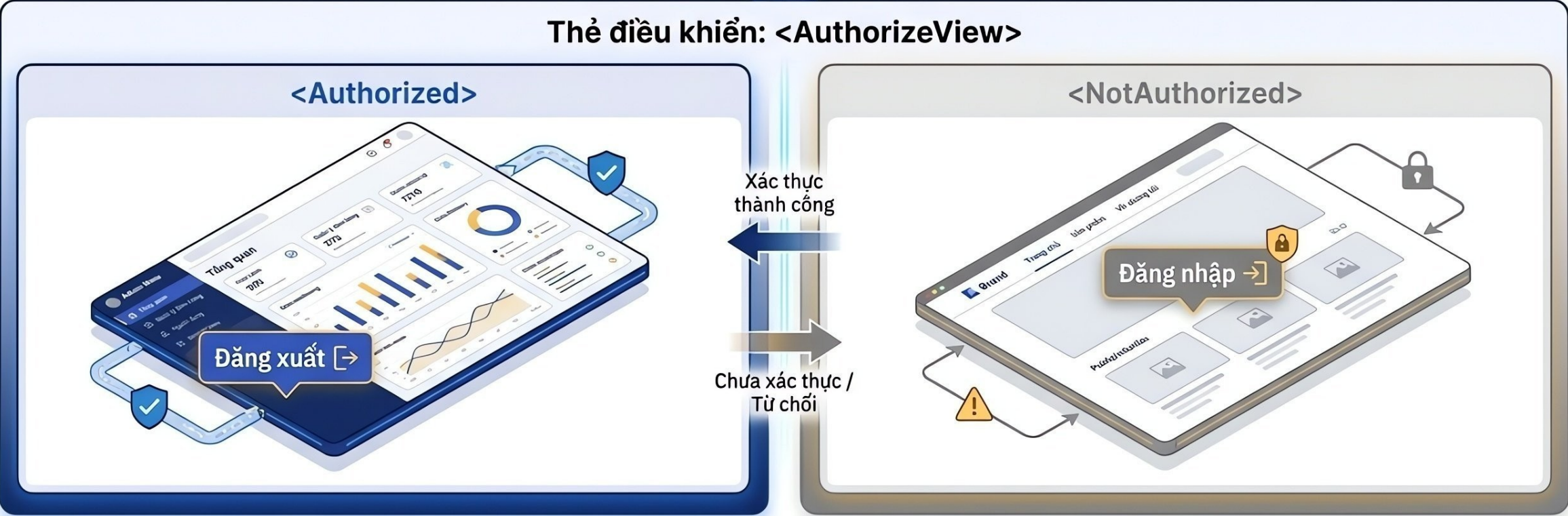


Khóa tuyến đường và điều khiển giao diện với AuthorizeView



@attribute [Authorize]

Đặt ở đầu trang Razor (@page "/admin/orders") để chặn truy cập từ mức URL.



Admin

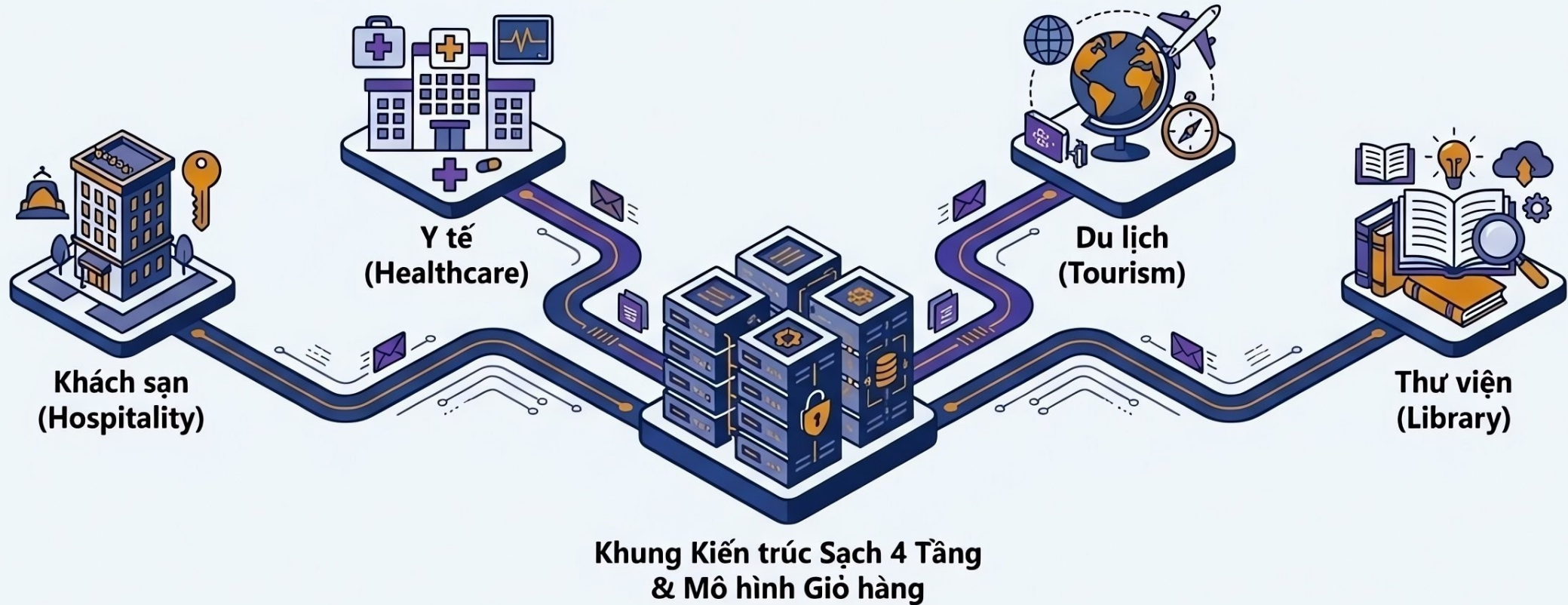
Admin

Phân quyền nâng cao theo vai trò: <AuthorizeView Roles="Admin,Manager">

Manager

Manager

Nguyên lý chuyển dịch: Từ lõi eShop sang 10 miền kinh doanh



Quy tắc Bắt buộc

Tuyệt đối KHÔNG nộp lại nguyên xi dự án eShop.
Mỗi nhóm (2-3 sinh viên) phải chọn 1 miền nghiệp vụ độc bản và tự thiết kế lại dữ liệu.



Nguyên lý Chuyển dịch (Abstraction Mapping)

Giữ nguyên 100% khung Kiến trúc Sạch. Thay thế toàn bộ Entity, thuộc tính nghiệp vụ, câu lệnh SQL và giao diện người dùng.

Bảng ma trận ánh xạ thực thể (Domain Transformation Matrix)

Lĩnh vực (Domain)	Product Node	Order Node	OrderLineItem Node	Category Node
Gốc: eShop Điện tử	Product	Order	OrderLineItem	Category
Đặt phòng Khách sạn	Room	Booking	BookingNight	RoomType
Khám bệnh Trực tuyến	Doctor (Service)	Appointment	[N/A]	Specialty
Bán Tour Du lịch	Tour	TourBooking	PassengerTicket	Destination
Quản lý Thư viện	Course (Book)	Enrollment	[N/A]	Topic

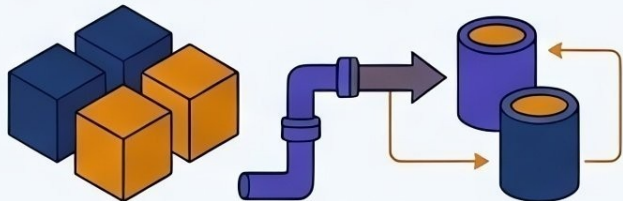
Bản chất luồng dữ liệu (UseCase & DataStore)
được ánh xạ 1:1, chỉ thay đổi ngôn ngữ nghiệp vụ.

Tiêu chuẩn đánh giá đồ án Capstone

(Cấu thành 60% điểm)

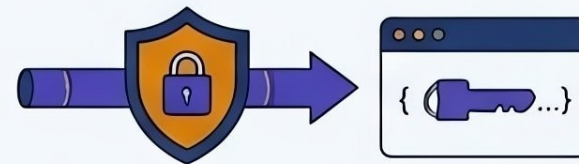
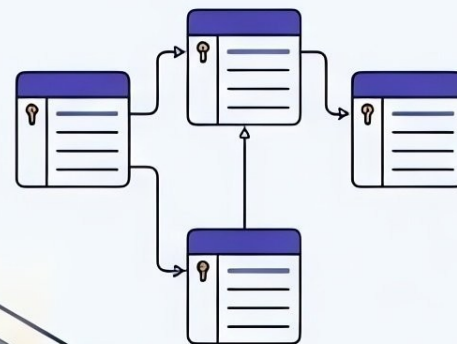
30% - Kiến trúc Sạch 4 Tầng

Phân chia đúng 4 Project C#;
Dependency Injection chuẩn mực,
không vi phạm nguyên lý DIP.



20% - CSDL SQL Server

CSDL chuẩn hóa tối thiểu 3NF;
truy vấn an toàn bằng
Parameterized Queries qua Dapper.



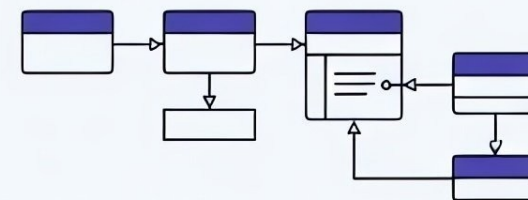
25% - Giao diện Blazor

Giao diện responsive; đầy đủ
tìm kiếm, giỏ hàng/đặt đơn,
form validation.

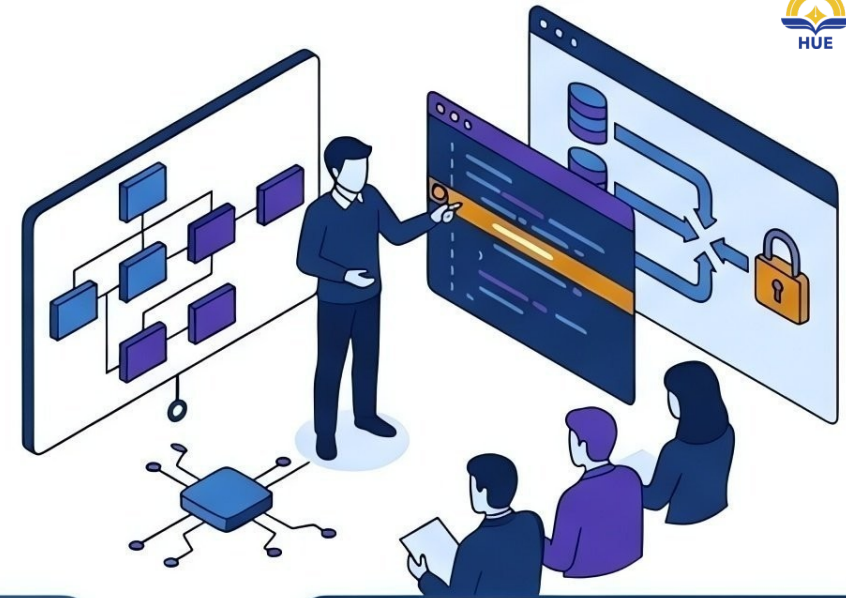


25% - Bảo vệ Vấn đáp

Giải thích rõ cơ chế vận hành
của từng dòng code và biểu đồ
UML.



Kỹ thuật giải trình kiến trúc: Làm chủ phiên bảo vệ vấn đáp



Bước 1: Khởi động

Trình chiếu Sơ đồ Kiến trúc 4 tầng & Sơ đồ Lớp trước khi mở ứng dụng.

Bước 2: Trình diễn

Demo luồng người dùng từ trang Khách hàng đến trang Quản trị Admin.

Bước 3: Giải mã

Mở file Program.cs giải thích cách đăng ký Service Lifetimes (Scoped).

Chiến thuật Phản biện

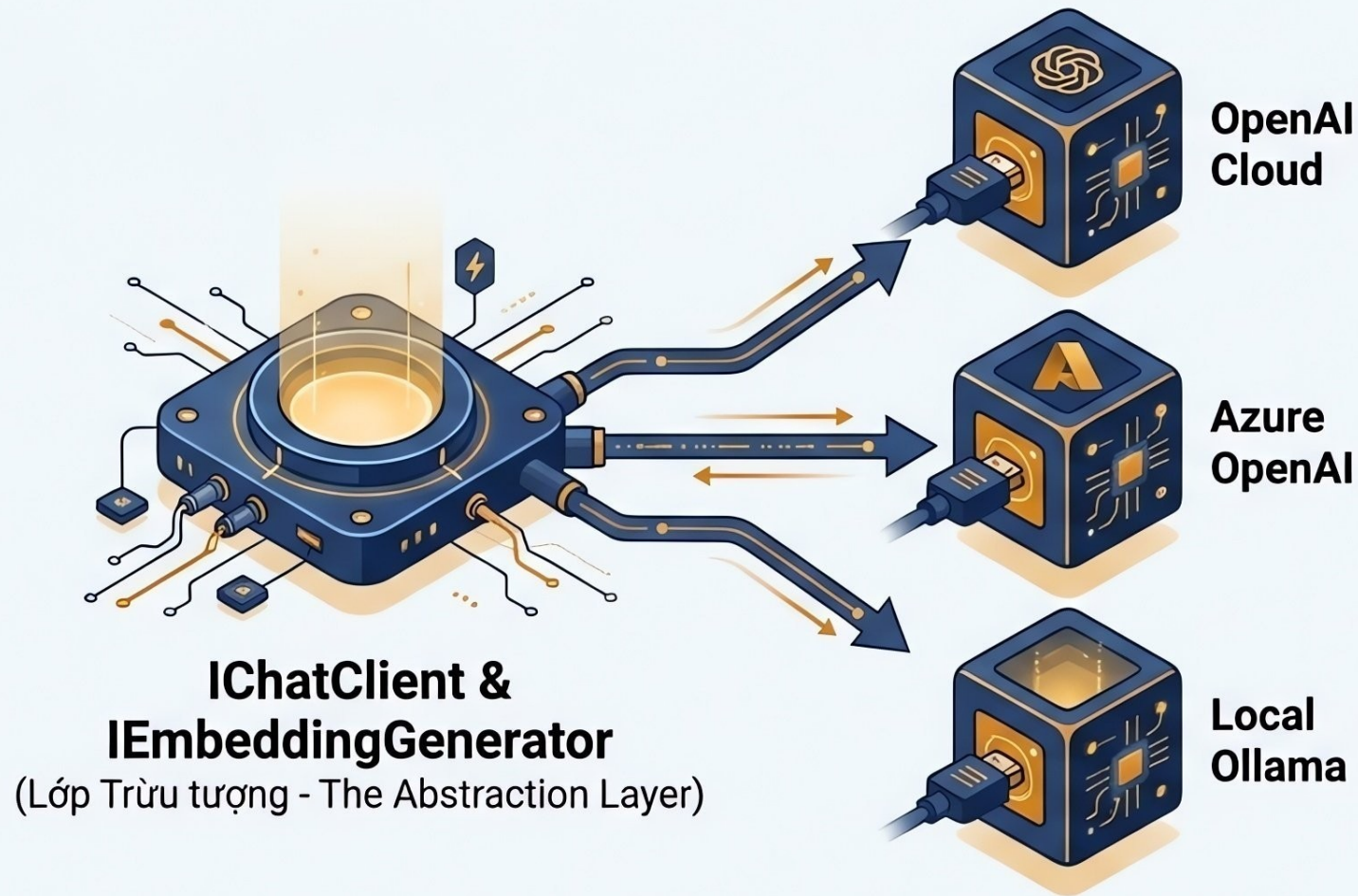


Luôn viện dẫn nguyên lý (DIP, Encapsulation) trước khi giải thích code.



Trực tiếp dùng Visual Studio đặt Breakpoint debug để chứng minh luồng chạy thực tế.

Kiến trúc cắm rút AI với Microsoft.Extensions.AI



Sức mạnh của Dependency Injection

1. **Chuẩn hóa giao tiếp:** Tích hợp liên mạch vào hệ thống của ASP.NET Core (Sẵn sàng cho 2025/2026).
2. **Lợi thế vô song:** Chuyển đổi nhà cung cấp AI chỉ qua 1 dòng cấu hình trong Program.cs — hoàn toàn không cần sửa đổi code nghiệp vụ.

Hiện thực hóa Trợ lý tư vấn AI (Shopping Assistant Agent)



Kiến trúc Triển khai

Tạo **ChatAssistantComponent.razor** nhúng ở góc giao diện.

Inject **IClient** vào một UseCase mới: **GetAiShoppingAdviceUseCase** (Độc lập, không phá vỡ logic cũ).

Vượt qua rào cản từ khóa với Semantic Search

Hạn chế: Tìm kiếm Từ khóa (LIKE)

Câu lệnh tìm kiếm:
"thiết bị gõ văn phòng êm ái
phòng êm ái"

Dữ liệu SQL:
"Bàn phím cơ Silent"

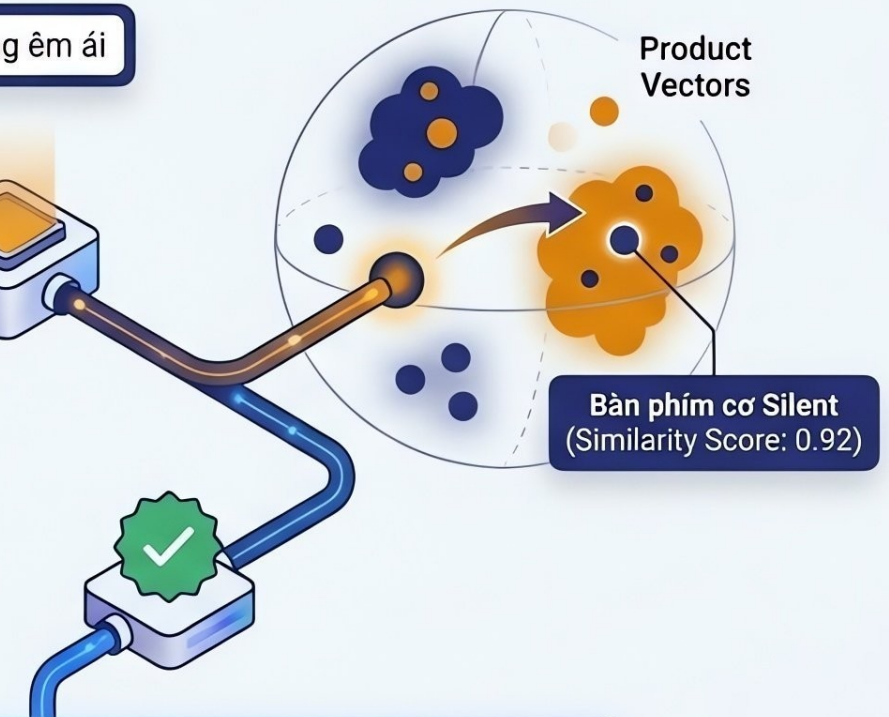


Đột phá: Tìm kiếm Ngữ nghĩa (Vector Embeddings)

thiết bị gõ văn phòng êm ái

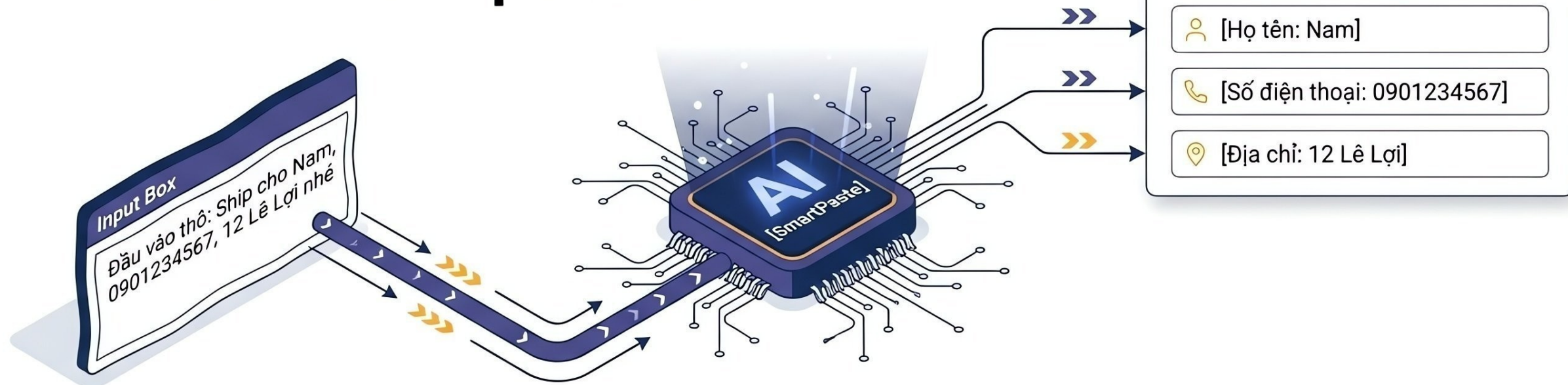
Vector
Embedding
Engine

Cơ chế: Chuyển đổi mô tả sản phẩm thành Vector Embeddings (mảng số thực đại diện ý nghĩa).



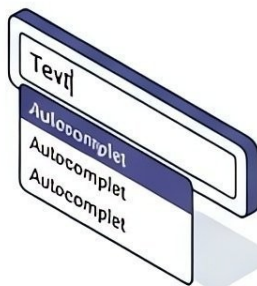
Kết quả: Thành công. So khớp độ tương đồng ngữ nghĩa –
Tìm đúng món hàng dù không trùng khớp chính xác từng chữ!

Tự động hóa biểu mẫu với Blazor Smart Components



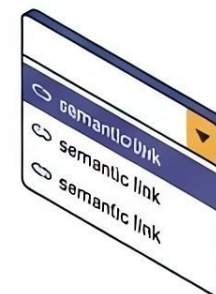
SmartTextArea

Tự động hoàn thành câu và gợi ý văn bản khi quản trị viên nhập mô tả sản phẩm.



SmartComboBox

Ô chọn lựa thông minh có khả năng hiểu các từ đồng nghĩa thay vì chỉ khớp chữ.



Quy trình đưa hệ thống vào thực tiễn: Docker & Cloud Hosting Schematic



Giai đoạn 1: Đóng gói Docker

Tạo Dockerfile đa giai đoạn. Tối ưu hóa dung lượng image < 150MB. Đảm bảo chạy đồng nhất trên mọi môi trường.



Giai đoạn 2: Tối ưu Biên dịch

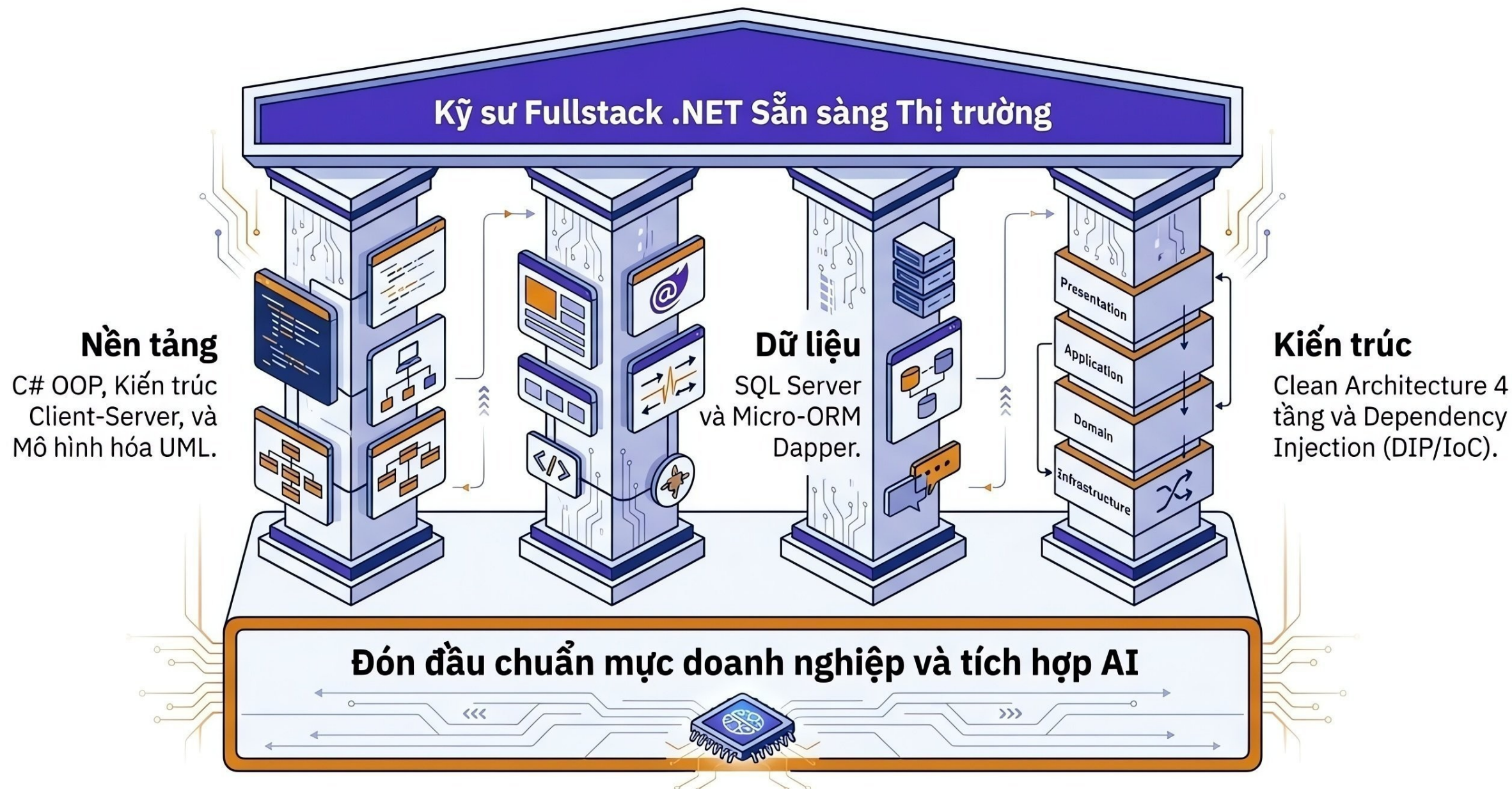
Biên dịch Native AOT — Giúp ứng dụng khởi động tức thì và giảm thiểu đáng kể tiêu hao bộ nhớ RAM.



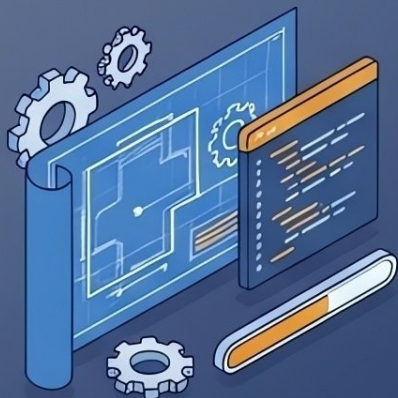
Giai đoạn 3: Triển khai Máy chủ

Hosting linh hoạt trên máy chủ Windows Server nội bộ hoặc các nền tảng Cloud (Azure App Service, AWS).

Tổng kết hành trình: Kiến tạo kỹ sư Fullstack .NET thực chiến



Kế hoạch hành động & Lộ trình phát triển nghề nghiệp



Kế hoạch Capstone Trước mắt

- Hoàn thiện đề cương và triển khai lập trình nhóm.
- Tham khảo cẩm nang Bridge-Guide.md để xử lý các lỗi kỹ thuật phát sinh.

Định hướng Nghề nghiệp Kỹ sư

- Backend .NET Developer tại các Ngân hàng, Fintech, Tập đoàn đa quốc gia.
- Fullstack Web Engineer hoặc Chuyên viên Tư vấn Giải pháp Hệ thống (ERP/CRM).



Code sạch hôm nay — Kiến tạo tương lai ngày mai!